Proyecto Final Programación III

Sistema Distribuido de Coordinación y Gestión de Emergencias en Elixi

Luisa Fernanda Salazar González

Código 1094888121

Daniela Naranjo Sanchez

Código 1010182628

Alejandro García Cadavid

Código 1095551008



Profesor: Julián Esteban Gutiérrez

Universidad del Quindío

Facultad de Ingeniería

Programa Ingeniería de Sistemas y Computación

Programación III

Tabla de contenido

1.1 Equipo de trabajo 2
1.2 Enunciado del proyecto 2
2. Desarrollo del proyecto final 3
2.1 Diagrama de arquitectura de módulos 3
2.2 Modelo de datos 4
2.3 Diagrama de secuencia 4
2.4 Diagrama de procesos y supervisión (OTP Tree) 5
3. Estructuración del proyecto 5
3.1 Estructura del proyecto 5
4. Demostración 6
4.1 Video de demostración 6
5. Repositorio 6
5.1 Link del repositorio 6
Lista de verificación final 7

1.1 Equipo de trabajo

Nombres completos	Código	Correo
Daniela Naranjo Sanchez	1010182628	daniela.naranjos@uqvirtual.edu.co
Alejandro Garcia	1095551008	alejandro.garciac@uqvirtual.edu.co
Luisa Fernanda Salazar	109488121	luisaf.salazarg@uqvirtual.edu.co

1.2 Enunciado del proyecto

Sistema multiusuario en Elixir que simula una plataforma distribuida para coordinar la atención después de un terremoto. Permite registrar y consultar usuarios, zonas afectadas, necesidades

puntuales, recursos, albergues, personas afectadas, mensajes entre familiares y reportes de mascotas perdidas o encontradas.

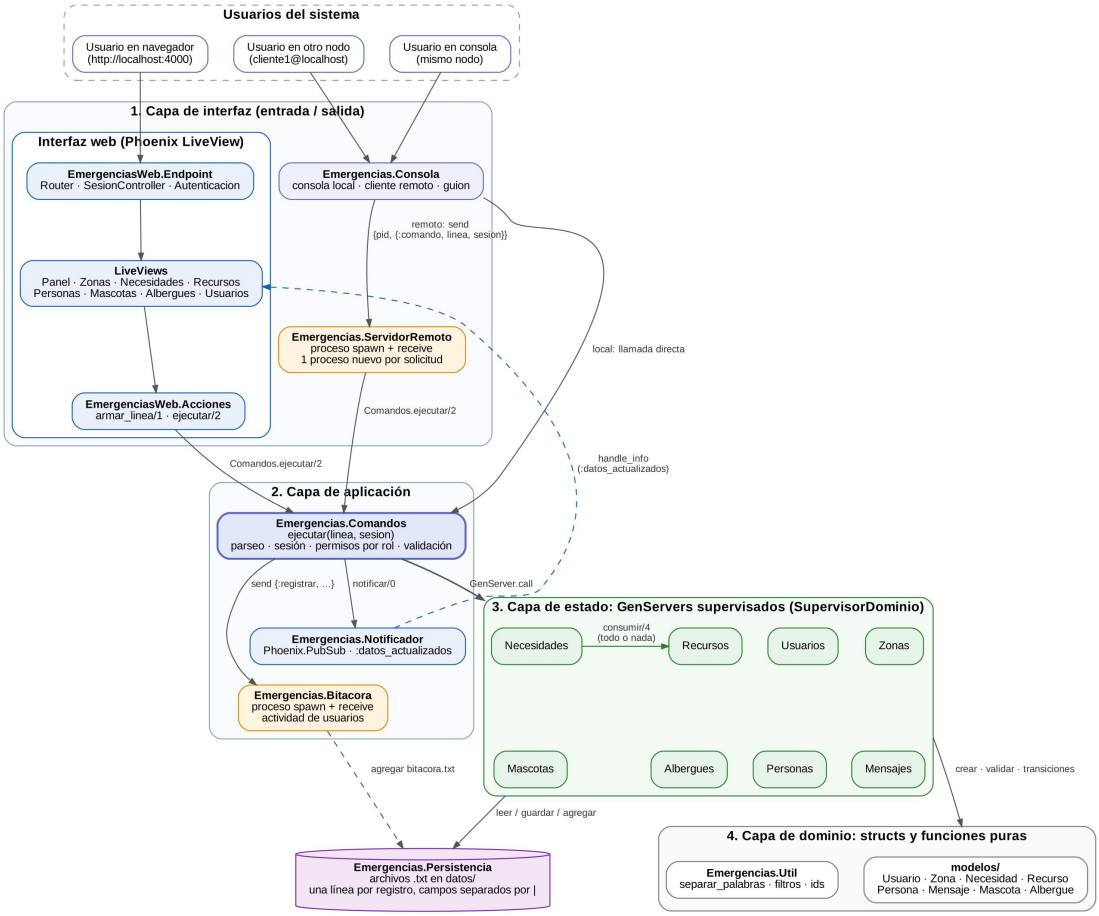
Los componentes trabajan de forma concurrente: el estado del dominio vive en 8 GenServers supervisados, la bitácora y el servidor remoto son procesos spawn/receive, y la información se conserva en archivos de texto plano. El sistema garantiza que, si dos solicitudes compiten por el mismo recurso, nunca se asigne más de lo que hay en inventario.

El sistema se usa de tres formas que comparten la misma lógica (Emergencias.Comandos): consola local, clientes en otros nodos de Erlang e interfaz web con Phoenix LiveView.

2. Desarrollo del proyecto final

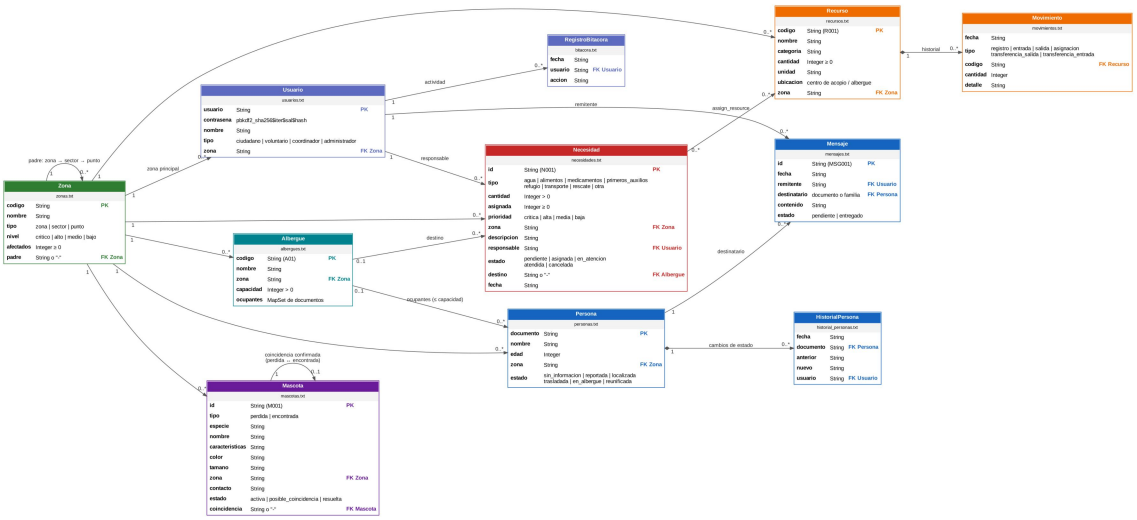
2.1 Diagrama de arquitectura de módulos

El sistema se organiza en 4 capas. Las tres interfaces (consola, cliente remoto y web) convierten lo que escribe el usuario en una línea de comando y la envían a Emergencias.Comandos. Comandos valida la sesión y el rol, registra la actividad en la bitácora y llama a los GenServers. Los GenServers usan los modelos (funciones puras) y guardan en disco a través de Persistencia, que es el único módulo que lee y escribe archivos.



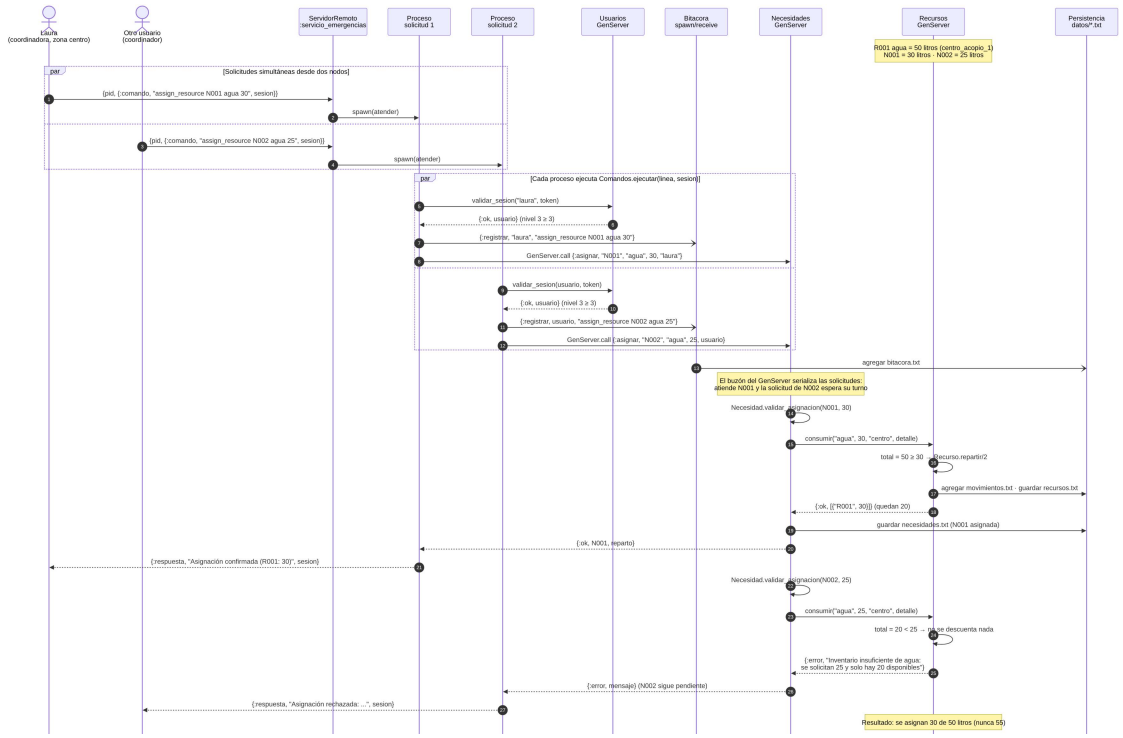
2.2 Modelo de datos

Cada entidad es un `struct` en `lib/emergencias/modelos/` y se guarda en su propio archivo `.txt`, una línea por registro con campos separados por `|`. La zona es jerárquica (zona → sector → punto de atención mediante el campo `padre`), y ese árbol se recorre de forma recursiva para calcular estadísticas acumuladas. Los historiales (movimientos de recursos, cambios de estado de personas y bitácora) solo se agregan, nunca se reescriben.



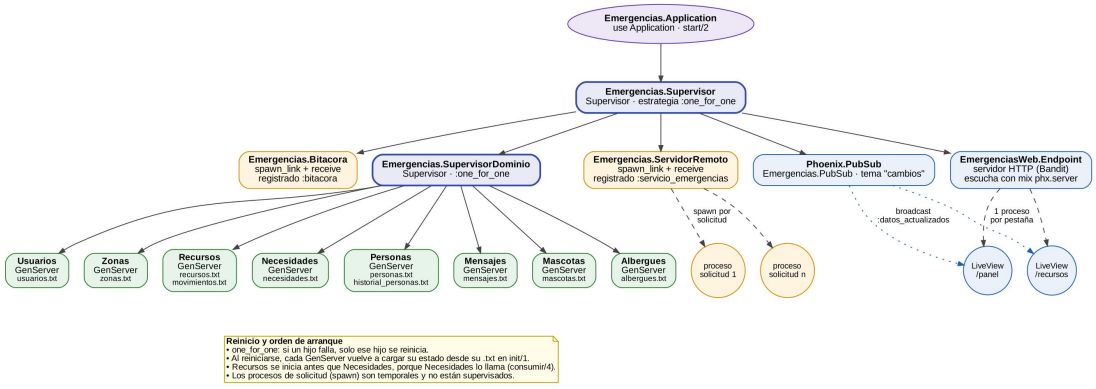
2.3 Diagrama de secuencia

El diagrama muestra el conflicto concurrente del flujo de ejemplo: hay 50 litros de agua (R001), la necesidad N001 pide 30 y, al mismo tiempo, N002 pide 25. Cada solicitud llega desde un nodo distinto y se atiende en su propio proceso. El GenServer `Necesidades` atiende una solicitud a la vez: confirma N001 (quedan 20 litros) y rechaza N002 sin descontar nada, así que nunca se asignan 55 litros.



2.4 Diagrama de procesos y supervisión (OTP Tree)

Emergencias.Supervisor usa la estrategia :one_for_one y tiene 5 hijos: la bitácora, SupervisorDominio (con los 8 GenServers), el servidor remoto, Phoenix.PubSub y el endpoint web. Si un GenServer falla, solo ese se reinicia y vuelve a cargar su estado desde su archivo. Los procesos que atienden cada solicitud remota son temporales y no se supervisan.



3. Estructuración del proyecto

3.1 Estructura del proyecto

El proyecto es una aplicación Mix. Separa las responsabilidades así: modelos/ contiene funciones puras que retornan {:ok, valor} o {:error, mensaje}; servidores/ guarda el estado y atiende solicitudes concurrentes; persistencia.ex es el único módulo con acceso a disco; comandos.ex

traduce el texto del usuario en llamadas a los servidores; y `emergencias_web/` es la interfaz web, sin lógica de negocio propia.

Estructura del proyecto · ProyectoFinalProgramacionIII (Elixir / Mix)

```
ProyectoFinalProgramacionIII/
├── mix.exs  # proyecto, dependencias y versión de Elixir
├── config/
│   └── config.exs  # carpeta de datos: datos/ o datos_test/
├── lib/
│   ├── emergencias.ex  # atajos: consola/0, cliente/1, guion/1
│   ├── emergencias/  # núcleo del sistema (sin dependencias externas)
│   │   ├── application.ex  # arranque y supervisor principal
│   │   ├── supervisor_dominio.ex  # supervisor de los 8 GenServers
│   │   ├── bitacora.ex  # proceso spawn/receive: actividad de usuarios
│   │   ├── servidor_remoto.ex  # proceso spawn/receive: clientes en otros nodos
│   │   ├── comandos.ex  # intérprete de comandos, sesión y permisos
│   │   ├── consola.ex  # interfaz de texto: local, remota y guion
│   │   ├── persistencia.ex  # único módulo que lee y escribe archivos
│   │   ├── notificador.ex  # avisa cambios a la web (Phoenix.PubSub)
│   │   ├── util.ex  # separar comandos, filtros, ids, números
│   │   ├── modelos/  # structs + funciones PURAS
│   │   │   ├── usuario.ex · zona.ex · necesidad.ex · recurso.ex
│   │   │   └── persona.ex · mensaje.ex · mascota.ex · albergue.ex
│   │   └── servidores/  # GenServers: estado + concurrencia
│   │       ├── usuarios.ex · zonas.ex · necesidades.ex · recursos.ex
│   │       └── personas.ex · mensajes.ex · mascotas.ex · albergues.ex
│   ├── emergencias_web.ex  # interfaz web opcional (Phoenix LiveView)
│   └── emergencias_web/
│       ├── endpoint.ex · router.ex  # entrada HTTP y rutas
│       ├── autenticacion.ex · acciones.ex  # sesión en LiveView y puente a Comandos
│       ├── controladores/  # inicio y cierre de sesión
│       ├── componentes/  # layouts y componentes visuales
│       └── live/  # panel, zonas, necesidades, recursos, personas, mascotas, albergues, usuarios
├── priv/static/css/app.css  # estilos de la interfaz web
├── test/  # pruebas ExUnit
│   ├── modelos_test · util_test · zona_test · comandos_test
│   └── concurrencia_test · interfaz_web_test · test_helper
├── guiones/flujo_ejemplo.txt  # flujo de ejemplo del enunciado
├── docs/  # diagramas (PNG) y fuentes
├── datos/  # persistencia en texto plano (campos separados por |)
│   ├── usuarios · zonas · necesidades · recursos · movimientos · personas
│   └── historial_personas · mensajes · mascotas · albergues · bitacora (.txt)
├── Proyecto Final Programacion.md  # enunciado del proyecto
└── README.md  # documentación del proyecto
```

4. Demostración

4.1 Video de demostración

Link del video:

5. Repositorio

5.1 Link del repositorio

https://github.com/fsg11/ProyectoFinalProgramacionIII

Lista de verificación final

Requisito	Cumple	Dónde se evidencia
El sistema gestiona usuarios	Sí	connect / disconnect, registro automático, 4 roles con permisos (servidores/usuarios.ex)
El sistema gestiona zonas y sectores	Sí	register_zone, zone_status con el árbol zona → sector → punto (modelos/zona.ex)
El sistema gestiona necesidades	Sí	register_need, list_needs con filtros, pending_needs ordenado por prioridad
El sistema gestiona recursos	Sí	register_resource, entradas y salidas, transfer_resource, historial en movimientos.txt
Existe comunicación entre familiares	Sí	report_person, find_person, send_family_message, historial de estados
Existe gestión de mascotas perdidas y encontradas	Sí	report_pet, match_pets con puntaje de similitud (umbral 60), confirm_match
Existe gestión de albergues	Sí	register_shelter, register_entry / register_exit, reporte de ocupación
Se utilizan varias estructuras de datos justificadas	Sí	Structs, mapas, listas, tuplas, MapSet y árbol anidado (README §4.1)
Existen al menos cuatro funciones recursivas propias	Sí	Zona.construir_arbol/2, Zona.total_afectados/1, Recurso.repartir/2, Util.separar/4, entre otras (README §4.2)
Existen procesos concurrentes	Sí	ServidorRemoto crea un proceso por solicitud; Bitacora es un proceso spawn/receive
Se utiliza GenServer	Sí	8 GenServers en lib/emergencias/servidores/
Existe Supervisor y/o DynamicSupervisor	Sí	Emergencias.Supervisor y SupervisorDominio, ambos :one_for_one
Se demuestra un conflicto concurrente resuelto correctamente	Sí	Diagrama 2.3, comando simulate_conflict y test/concurrencia_test.exs
Existe persistencia	Sí	Archivos .txt en datos/ (persistencia.ex)
Existen pruebas	Sí	7 archivos en test/, ejecutados con mix test
Los diagramas corresponden al código final	Sí	Diagramas de las secciones 2 y 3, generados a partir del código entregado

Requisito	Cumple	Dónde se evidencia
Todos los integrantes participan en el repositorio y la sustentación	Sí	Commits de cada integrante en el repositorio